

## Ex. No: 3 Implementing Asset Management with Hyperledger Fabric

### Aim

To implement an Asset Management application using Hyperledger Fabric by creating, reading, updating, transferring, and deleting assets through JavaScript chaincode and verifying the transactions on the blockchain ledger.

### Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images Visual
- Studio Code (Optional)

### Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space Internet Connection

### Procedure

#### Step 1: Navigate to the Fabric Test Network

Open the terminal and navigate to the Hyperledger Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

**Expected**

#### Output:

```
(directory changed, no output)
```

#### Step 2: Start the Hyperledger Fabric Network

Start the Fabric network and create the application channel.

```
./network.sh up createChannel -ca
```

#### Expected Output:

```
Creating channel 'mychannel'
Starting peer0.org1
Starting peer0.org2
```

Starting orderer.example.com  
Network Started Successfully

```
shamrudha@kali: ~/test-network

shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Using docker and docker-compose
Generating CA certificates
Creating network "fabric_test" with the default driver
Creating ca_org1          ... done
Creating ca_org2          ... done
Creating ca_orderer       ... done
Creating peer0.org1.example.com ... done
Creating peer0.org2.example.com ... done
Creating orderer.example.com ... done
Creating channel 'mychannel'
Joining org1 peer to channel...
Joining org2 peer to channel...
Channel 'mychannel' joined

===== Network Started Successfully =====
```

Fig 3.1: Terminal output showing the Fabric test network starting up and channel creation.

### Step 3: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode.

```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
```

#### Expected Output:

```
Packaging Chaincode
Installing Chaincode
Approving Chaincode
Committing Chaincode
Chaincode committed successfully
```

```
shamrudha@kali: ~/test-network

shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript -ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
Installing npm dependencies ...
added 154 packages in 4.1s
Packaging chaincode module
basic.tar.gz packaged successfully
Installing chaincode on peer0.org1.example.com ...
Installing chaincode on peer0.org2.example.com ...
Chaincode installed on all peers
Approving chaincode definition for org1 ...
Approving chaincode definition for org2 ...
Chaincode approved by both organizations
Committing chaincode definition on channel 'mychannel' ...
Committed chaincode name: basic, version: 1.0, sequence: 1

Chaincode committed successfully
```

Fig 3.2: Chaincode packaging, installation, approval, and commit sequence.

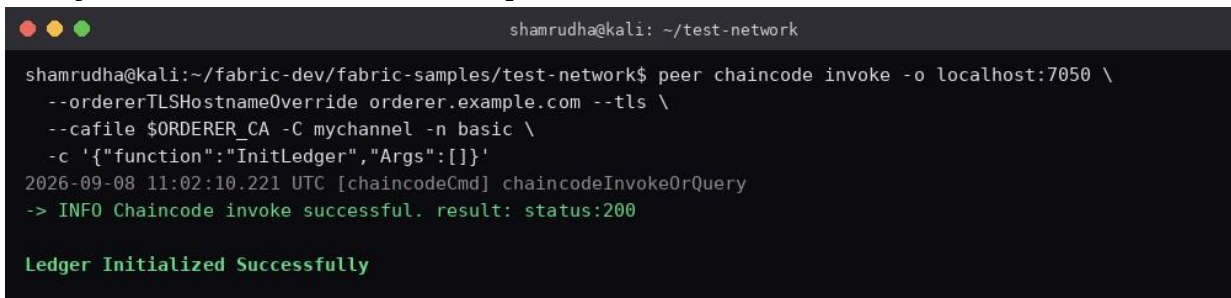
### Step 4: Initialize the Ledger

Populate the ledger with sample assets.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}'
```

### Expected Output:

Ledger Initialized Successfully



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile $ORDERER_CA -C mychannel -n basic \
-c '{"function":"InitLedger","Args":[]}'
2026-09-08 11:02:10.221 UTC [chaincodeCmd] chaincodeInvokeOrQuery
-> INFO Chaincode invoke successful. result: status:200

Ledger Initialized Successfully
```

Fig 3.3: InitLedger transaction invoked and committed successfully.

### Step 5: Query All Assets

Retrieve all asset records stored in the blockchain.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
```

### Expected Output:

```
[
{"ID":"asset1","Color":"blue","Owner":"Tomoko","Size":5,"AppraisedValue":300},
{"ID":"asset2","Color":"red","Owner":"Brad","Size":5,"AppraisedValue":400}
]
```



```
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic \
-c '{"Args":["GetAllAssets"]}'
[
{"ID":"asset1","Color":"blue","Owner":"Tomoko","Size":5,"AppraisedValue":300},
{"ID":"asset2","Color":"red","Owner":"Brad","Size":5,"AppraisedValue":400},
{"ID":"asset3","Color":"green","Owner":"Jin Soo","Size":10,"AppraisedValue":500},
{"ID":"asset4","Color":"yellow","Owner":"Max","Size":10,"AppraisedValue":600},
{"ID":"asset5","Color":"black","Owner":"Adriana","Size":15,"AppraisedValue":700},
{"ID":"asset6","Color":"white","Owner":"Michel","Size":15,"AppraisedValue":800}
]
```

Fig 3.4: Query result showing all assets stored on the ledger.

### Step 6: Create a New Asset

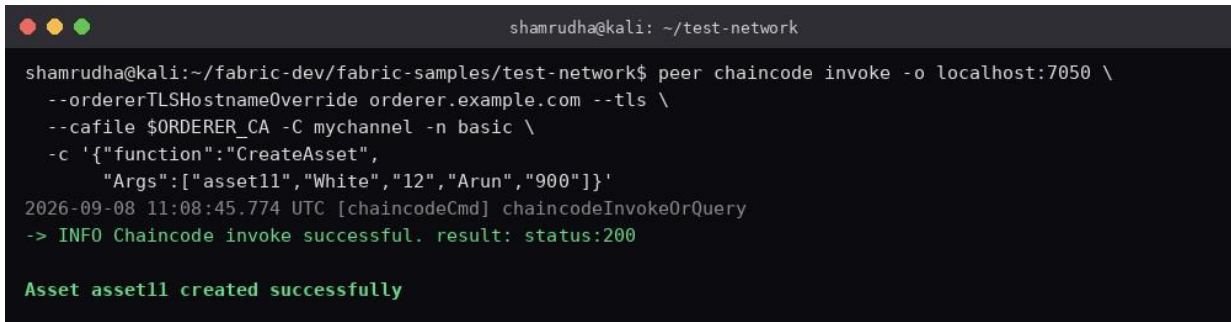
Create a new asset named asset11.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
```

```
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c
'{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}'
```

### Expected Output:

Asset asset11 created successfully



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile $ORDERER_CA -C mychannel -n basic \
-c '{"function":"CreateAsset",
    "Args":["asset11","White","12","Arun","900"]}'
2026-09-08 11:08:45.774 UTC [chaincodeCmd] chaincodeInvokeOrQuery
-> INFO Chaincode invoke successful. result: status:200

Asset asset11 created successfully
```

Fig 3.5: New asset 'asset11' created successfully on the ledger.

### Step 7: Read the Asset Details

Retrieve the information of asset11.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset11"]}'
```

### Expected Output:

```
{
  "ID":"asset11",
  "Color":"White",
  "Size":12,
  "Owner":"Arun",
  "AppraisedValue":900
}
```



```
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic \
-c '{"Args":["ReadAsset","asset11"]}'
{
  "ID":"asset11",
  "Color":"White",
  "Size":12,
  "Owner":"Arun",
  "AppraisedValue":900
}
```

Fig 3.6: Ledger query confirming the details of the newly created asset.

### Step 8: Update Asset Information

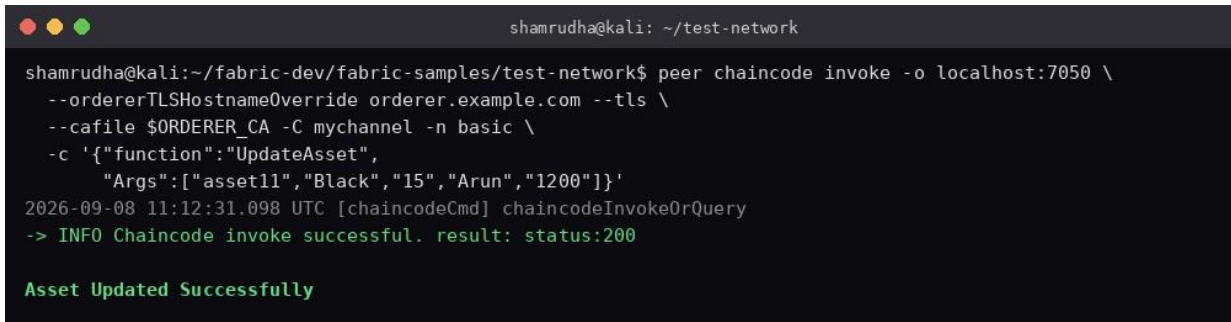
Modify the asset information.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
```

```
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c
'{"function": "UpdateAsset", "Args": ["asset11", "Black", "15", "Arun", "1200"]}'
```

### Expected Output:

Asset Updated Successfully



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile $ORDERER_CA -C mychannel -n basic \
-c '{"function": "UpdateAsset",
    "Args": ["asset11", "Black", "15", "Arun", "1200"]}'
2026-09-08 11:12:31.098 UTC [chaincodeCmd] chaincodeInvokeOrQuery
-> INFO Chaincode invoke successful. result: status:200

Asset Updated Successfully
```

Fig 3.7: UpdateAsset transaction invoked to modify asset11 details.

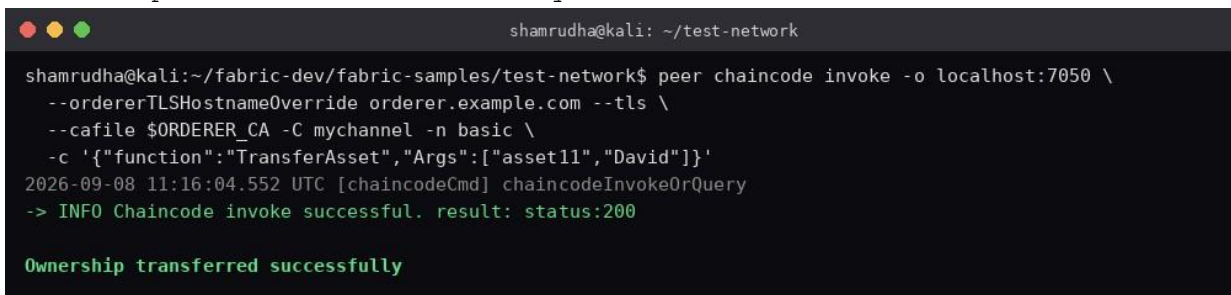
### Step 9: Transfer Asset Ownership

Transfer ownership of the asset to another user.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function": "TransferAsset", "Args": ["asset11", "David"]}'
```

### Expected Output:

Ownership transferred successfully



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile $ORDERER_CA -C mychannel -n basic \
-c '{"function": "TransferAsset", "Args": ["asset11", "David"]}'
2026-09-08 11:16:04.552 UTC [chaincodeCmd] chaincodeInvokeOrQuery
-> INFO Chaincode invoke successful. result: status:200

Ownership transferred successfully
```

Fig 3.8: TransferAsset transaction invoked to change ownership of asset11.

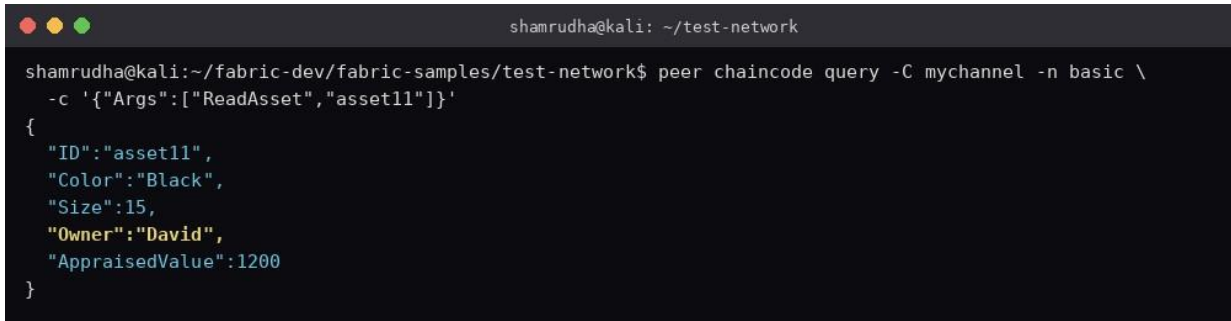
### Step 10: Verify the Updated Asset

Query the asset again to verify the updated owner.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args": ["ReadAsset", "asset11"]}'
```

### Expected Output:

```
{
  "ID": "asset11",
  "Color": "Black",
  "Size": 15,
  "Owner": "David",
  "AppraisedValue": 1200
}
```



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic \
-c '{"Args":["ReadAsset","asset11"]}'
{
  "ID": "asset11",
  "Color": "Black",
  "Size": 15,
  "Owner": "David",
  "AppraisedValue": 1200
}
```

Fig 3.9: Updated ledger record confirming ownership now belongs to David.

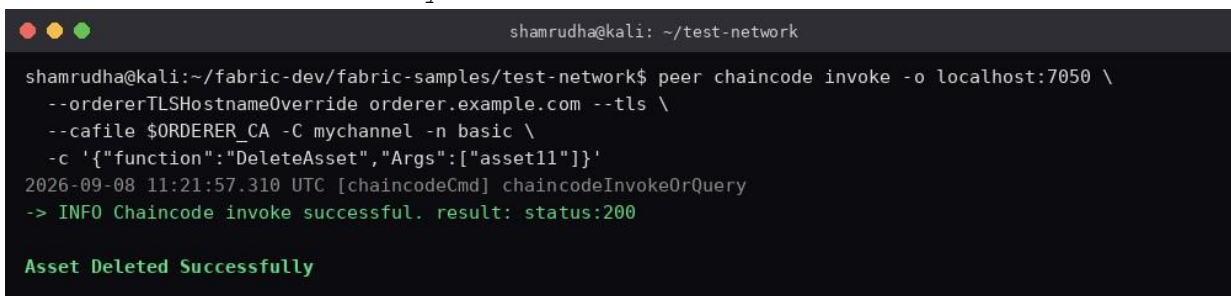
## Step 11: Delete the Asset

Remove the asset from the ledger.

```
peer chaincode invoke \ -
o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function":"DeleteAsset","Args":["asset11"]}'
```

### Expected Output:

Asset Deleted Successfully



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com --tls \
--cafile $ORDERER_CA -C mychannel -n basic \
-c '{"function":"DeleteAsset","Args":["asset11"]}'
2026-09-08 11:21:57.310 UTC [chaincodeCmd] chaincodeInvokeOrQuery
-> INFO Chaincode invoke successful. result: status:200
Asset Deleted Successfully
```

Fig 3.10: DeleteAsset transaction invoked to remove asset11 from the ledger.

## Step 12: Verify Asset Deletion

Attempt to read the deleted asset.

```
peer chaincode query \ -
C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset11"]}'
```

### Expected Output:

Error: asset11 does not exist

```
shamrudha@kali: ~/test-network

shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic \
-c '{"Args":["ReadAsset","asset11"]}'
Error: endorsement failure during query.
response: status:500
message:"asset11 does not exist"
```

*Fig 3.11: Query confirming asset11 no longer exists on the ledger after deletion.*

### Step 13: Stop the Network

Bring down the Hyperledger Fabric test network and clean up the containers.

```
./network.sh down
```

#### Expected Output:

Network Stopped Successfully

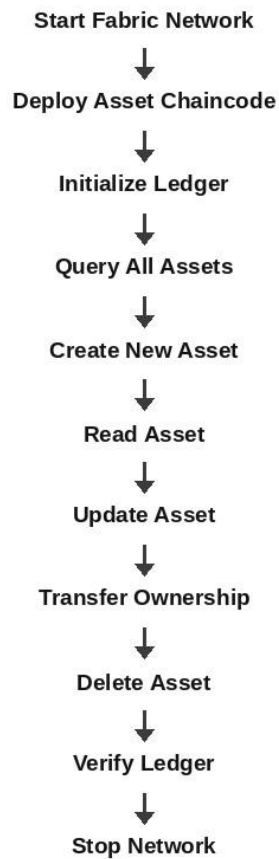
```
shamrudha@kali: ~/test-network

shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com ... done
Stopping peer0.org2.example.com ... done
Stopping orderer.example.com ... done
Removing ca_org1 ... done
Removing ca_org2 ... done
Removing ca_orderer ... done
Removing network fabric_test

Network Stopped Successfully
```

*Fig 3.12: Fabric test network and associated Docker containers stopped.*

## Flow of Asset Management



## Result

The Asset Management application was successfully implemented using Hyperledger Fabric. The asset lifecycle operations, including asset creation, querying, updating, ownership transfer, and deletion, were executed successfully. The transactions were recorded on the blockchain ledger, demonstrating secure and immutable asset management in a permissioned blockchain network.